

Warp GPU 布料仿真程序梳理与 VBD 数学推导

A Structured Academic Summary of Vertex Block Descent for Cloth Simulation

文档定位：本文面向仿真开发与数值算法实现，围绕用户提供的可运行 Warp 程序，系统总结其数据结构、GPU 执行流程、质量-弹簧布料模型、隐式 Euler 离散、VBD 局部牛顿更新以及 ParaView 输出变量。

Prepared for: 金明亮

Date: 2026-06-02

Style: restrained academic layout with Hong Kong university-inspired navy, burgundy and gold accents

Contents

1 执行摘要 2

2 程序结构与数据流 2

3 布料几何离散 3

3.1 固定边界条件 3

4 质量-弹簧布料模型 3

4.1 集中质量 3

4.2 三类弹簧 3

5 连续动力学方程 4

6 隐式 Euler 离散与能量形式 4

7 VBD 局部优化问题 5

8 弹簧梯度推导 5

9 弹簧 Hessian 推导 6

10 单顶点梯度与 Hessian 6

11 局部牛顿步与 3×3 求解 7

12 步长限制与数值稳定性 7

13 9 色并行 Gauss-Seidel 8

14 速度回算与阻尼 8

15 ParaView 输出变量说明 8

16 算法优点与局限 8

16.1 优点 9

16.2 局限 9

17 最终公式总览 10

结语 10

1 执行摘要

该程序实现的是一个基于 **NVIDIA Warp** 的 GPU 布料仿真框架。物理模型采用质量-弹簧系统，时间离散采用隐式 Euler，非线性隐式问题通过 **VBD(Vertex Block Descent)** 分解为逐顶点的三维局部牛顿问题。程序最终输出 OBJ、VTP、PVD 和可选 USD 文件，其中 VTP/PVD 更适合在 ParaView 中播放动画和查看速度、位移、固定点、VBD 颜色等点数据。

核心结论

一个时间子步的计算可以概括为：

预测惯性点 y_i $\implies$ VBD 逐顶点局部牛顿迭代 $\implies$ 由位移差回算速度

VBD 的核心更新公式为：

$$\mathbf{x}_i^{\text{new}} = \mathbf{x}_i - \mathbf{H}_i^{-1} \mathbf{g}_i$$

其中 $\mathbf{g}_i$ 是局部能量对顶点 i 的梯度， $\mathbf{H}_i$ 是对应的 3×3 Hessian 或半正定近似 Hessian。

2 程序结构与数据流

程序从功能上可分为五个模块：

Table 1: 程序模块梳理

模块	主要作用
CPU 网格生成	生成规则方形布料顶点、固定点、三角面片和初始扰动。
弹簧拓扑构建	建立结构弹簧、剪切弹簧和两跳弯曲弹簧，并写入定长邻接表。
GPU 预测步	根据旧位置、速度和重力计算隐式 Euler 的惯性中心 y_i 。
GPU VBD 求解	对每个顶点构造局部能量、梯度和 Hessian，执行颜色分组的局部牛顿更新。
输出与可视化	输出 OBJ 单帧几何，VTP/PVD 时间序列，以及可选 USD 动画。

主循环逻辑可以表达为：

```
for frame in range(1, frames + 1):
    for substep in range(substeps):
        predict_kernel(...)
        for iter in range(vbd_iters):
            for color in range(9):
                vbd_color_kernel(...)
        finalize_kernel(...)
    save_obj_vtp_pvd_if_needed(...)
```

这种结构体现了典型的 GPU 仿真分工：CPU 负责拓扑、参数和文件输出；GPU 负责大规模顶点并行计算。

3 布料几何离散

设方形布料边长为 L ，每边分辨率为 n ，则顶点总数为：

$$N = n^2.$$

网格间距为：

$$\Delta x = \frac{L}{n-1}.$$

二维网格索引 (i_x, i_y) 映射为一维编号：

$$\text{id}(i_x, i_y) = i_y n + i_x.$$

初始顶点位置近似为：

$$\mathbf{X}_{i_x, i_y} = \begin{bmatrix} \left(\frac{i_x}{n-1} - \frac{1}{2} \right) L \\ -i_y \Delta x \\ 0.008 \Delta x \sin(1.73 i_x + 2.41 i_y) \end{bmatrix}.$$

其中第三个分量的小扰动用于打破完全平面对称性，使布料在自然下垂时更容易形成褶皱。

3.1 固定边界条件

程序固定上边缘的两个角点：

$$\mathcal{P} = \{(0, 0), (n-1, 0)\}.$$

对于固定点，有：

$$\mathbf{x}_i(t) = \mathbf{X}_i^{\text{pin}}, \quad \mathbf{v}_i(t) = \mathbf{0}, \quad i \in \mathcal{P}.$$

这对应程序中的 `fixed[i]=1`。在预测和收尾 `kernel` 中，固定点都会被强制恢复到钉住位置。

4 质量-弹簧布料模型

程序采用质量-弹簧模型而非连续壳单元。每个顶点为集中质量点，每条弹簧连接一对顶点。

4.1 集中质量

设布料面密度为 ρ_s ，总面积为 $A = L^2$ ，则总质量为：

$$M_{\text{total}} = \rho_s A.$$

代码采用均匀集中质量：

$$m_i = m = \frac{\rho_s L^2}{N}.$$

4.2 三类弹簧

所有弹簧的总势能为：

$$U(\mathbf{x}) = \sum_{(i,j) \in \mathcal{E}} \frac{1}{2} k_{ij} (\|\mathbf{x}_i - \mathbf{x}_j\| - L_{ij})^2,$$

Table 2: 弹簧类型与物理作用

类型	连接方式	静止长度	作用
结构弹簧	水平/竖直相邻点	Δx	控制主方向拉伸刚度。
剪切弹簧	每个方格两条对角线	$\sqrt{2}\Delta x$	控制斜向剪切变形。
弯曲弹簧	横向/纵向跨两个网格点	$2\Delta x$	近似抑制局部弯曲。

其中 $\mathcal{E}$ 是弹簧边集合, k_{ij} 是弹簧刚度, L_{ij} 是静止长度。

关于弯曲弹簧的说明

程序中的弯曲弹簧是距离型的两跳弹簧, 用于近似提高弯曲刚度。它不是严格的离散壳二面角弯曲能量。如果后续目标是高保真布料或薄壳, 需要进一步引入三角形膜能量和二面角弯曲能量。

5 连续动力学方程

设所有自由顶点组成向量:

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_N \end{bmatrix}^T, \quad \mathbf{x}_i \in \mathbb{R}^3.$$

质量矩阵为块对角形式:

$$\mathbf{M} = \text{diag}(m_1 \mathbf{I}_3, m_2 \mathbf{I}_3, \dots, m_N \mathbf{I}_3).$$

忽略碰撞与空气阻力, 布料动力学方程为:

$$\mathbf{M} \ddot{\mathbf{x}} = -\nabla U(\mathbf{x}) + \mathbf{M} \mathbf{g}.$$

对单个顶点:

$$m_i \ddot{\mathbf{x}}_i = -\frac{\partial U}{\partial \mathbf{x}_i} + m_i \mathbf{g}.$$

6 隐式 Euler 离散与能量形式

隐式 Euler 格式为:

$$\mathbf{v}_i^{n+1} = \mathbf{v}_i^n + h \left(\mathbf{g} - \frac{1}{m_i} \frac{\partial U}{\partial \mathbf{x}_i}(\mathbf{x}^{n+1}) \right), \quad (1)$$

$$\mathbf{x}_i^{n+1} = \mathbf{x}_i^n + h \mathbf{v}_i^{n+1}. \quad (2)$$

将速度式代入位置式:

$$\mathbf{x}_i^{n+1} = \mathbf{x}_i^n + h \mathbf{v}_i^n + h^2 \mathbf{g} - \frac{h^2}{m_i} \frac{\partial U}{\partial \mathbf{x}_i}(\mathbf{x}^{n+1}).$$

定义惯性预测点:

$$\mathbf{y}_i = \mathbf{x}_i^n + h \mathbf{v}_i^n + h^2 \mathbf{g}.$$

则隐式 Euler 的一阶平衡条件变为：

$$\frac{m_i}{h^2} (\mathbf{x}_i^{n+1} - \mathbf{y}_i) + \frac{\partial U}{\partial \mathbf{x}_i}(\mathbf{x}^{n+1}) = \mathbf{0}.$$

这等价于最小化如下全局能量：

$$E(\mathbf{x}) = \sum_i \frac{m_i}{2h^2} \|\mathbf{x}_i - \mathbf{y}_i\|^2 + \sum_{(i,j) \in \mathcal{E}} \frac{1}{2} k_{ij} (\|\mathbf{x}_i - \mathbf{x}_j\| - L_{ij})^2$$

其中第一项是惯性项，第二项是弹簧势能项。

7 VBD 局部优化问题

VBD 不直接组装和求解全局稀疏矩阵，而是将全局能量拆解为逐顶点的局部优化问题。更新顶点 i 时，将邻居顶点 $j \in \mathcal{N}(i)$ 暂时视为已知，局部能量为：

$$E_i(\mathbf{x}_i) = \frac{\alpha_i}{2} \|\mathbf{x}_i - \mathbf{y}_i\|^2 + \sum_{j \in \mathcal{N}(i)} \frac{1}{2} k_{ij} (\|\mathbf{x}_i - \mathbf{x}_j\| - L_{ij})^2,$$

其中：

$$\alpha_i = \frac{m_i}{h^2}.$$

在代码中该量对应：

$$\text{inertia} = \frac{\text{vertex_mass}}{\text{sub_dt}^2}.$$

8 弹簧梯度推导

对单根弹簧 (i, j) ，定义：

$$\mathbf{d}_{ij} = \mathbf{x}_i - \mathbf{x}_j, \quad r_{ij} = \|\mathbf{d}_{ij}\|, \quad \mathbf{n}_{ij} = \frac{\mathbf{d}_{ij}}{r_{ij}}.$$

弹簧能量为：

$$\phi_{ij}(\mathbf{x}_i) = \frac{1}{2} k_{ij} (r_{ij} - L_{ij})^2.$$

由于：

$$\frac{\partial r_{ij}}{\partial \mathbf{x}_i} = \mathbf{n}_{ij},$$

所以：

$$\nabla_{\mathbf{x}_i} \phi_{ij} = k_{ij} (r_{ij} - L_{ij}) \mathbf{n}_{ij}.$$

进一步写成：

$$\nabla_{\mathbf{x}_i} \phi_{ij} = k_{ij} \left(1 - \frac{L_{ij}}{r_{ij}}\right) (\mathbf{x}_i - \mathbf{x}_j).$$

令：

$$s_{ij} = 1 - \frac{L_{ij}}{r_{ij}},$$

则:

$$\nabla_{\mathbf{x}_i} \phi_{ij} = k_{ij} s_{ij} \mathbf{d}_{ij}$$

这对应代码中的:

```
stretch = 1.0 - rest * inv_r
g += k * stretch * dx
```

9 弹簧 Hessian 推导

梯度为:

$$\mathbf{g}_{ij} = k_{ij} s_{ij} \mathbf{d}_{ij}.$$

对 $\mathbf{x}_i$ 再求导:

$$\mathbf{H}_{ij} = k_{ij} \left[s_{ij} \mathbf{I} + \frac{L_{ij}}{r_{ij}} \mathbf{n}_{ij} \mathbf{n}_{ij}^T \right].$$

因为:

$$1 - s_{ij} = \frac{L_{ij}}{r_{ij}},$$

所以也可写为:

$$\mathbf{H}_{ij} = k_{ij} \left[s_{ij} \mathbf{I} + (1 - s_{ij}) \mathbf{n}_{ij} \mathbf{n}_{ij}^T \right]$$

当弹簧压缩时 $r_{ij} < L_{ij}$, 有 $s_{ij} < 0$, 此时精确 Hessian 可能非正定。程序为增强稳定性采用:

$$s_{ij}^+ = \max(s_{ij}, 0),$$

并使用半正定近似:

$$\widetilde{\mathbf{H}}_{ij} = k_{ij} \left[s_{ij}^+ \mathbf{I} + (1 - s_{ij}^+) \mathbf{n}_{ij} \mathbf{n}_{ij}^T \right].$$

代码中的 tangent 裁剪即对应该处理:

```
tangent = stretch
if tangent < 0.0:
    tangent = 0.0
kt = k * tangent
kn = k * (1.0 - tangent)
H += kt * I + kn * n * n.T
```

10 单顶点梯度与 Hessian

局部能量的梯度为:

$$\mathbf{g}_i = \alpha_i (\mathbf{x}_i - \mathbf{y}_i) + \sum_{j \in \mathcal{N}(i)} k_{ij} \left(1 - \frac{L_{ij}}{\|\mathbf{x}_i - \mathbf{x}_j\|} \right) (\mathbf{x}_i - \mathbf{x}_j)$$

局部 Hessian 近似为:

$$\mathbf{H}_i = \alpha_i \mathbf{I} + \sum_{j \in \mathcal{N}(i)} k_{ij} \left[s_{ij}^+ \mathbf{I} + (1 - s_{ij}^+) \mathbf{n}_{ij} \mathbf{n}_{ij}^T \right]$$

其中:

$$\mathbf{n}_{ij} = \frac{\mathbf{x}_i - \mathbf{x}_j}{\|\mathbf{x}_i - \mathbf{x}_j\|}, \quad s_{ij}^+ = \max \left(1 - \frac{L_{ij}}{\|\mathbf{x}_i - \mathbf{x}_j\|}, 0 \right).$$

11 局部牛顿步与 3×3 求解

对 E_i 在当前 $\mathbf{x}_i$ 附近进行二阶近似:

$$E_i(\mathbf{x}_i + \Delta \mathbf{x}) \approx E_i(\mathbf{x}_i) + \mathbf{g}_i^T \Delta \mathbf{x} + \frac{1}{2} \Delta \mathbf{x}^T \mathbf{H}_i \Delta \mathbf{x}.$$

令增量的导数为零:

$$\mathbf{g}_i + \mathbf{H}_i \Delta \mathbf{x} = \mathbf{0}.$$

因此:

$$\Delta \mathbf{x} = -\mathbf{H}_i^{-1} \mathbf{g}_i.$$

更新为:

$$\mathbf{x}_i^{\text{new}} = \mathbf{x}_i - \mathbf{H}_i^{-1} \mathbf{g}_i$$

程序手写了对称 3×3 矩阵求逆。设:

$$\mathbf{H} = \begin{bmatrix} a & b & c \\ b & d & e \\ c & e & f \end{bmatrix}.$$

其伴随矩阵的独立元素为:

$$\begin{aligned} C_{00} &= df - e^2, & C_{01} &= ce - bf, & C_{02} &= be - cd, \\ C_{11} &= af - c^2, & C_{12} &= bc - ae, & C_{22} &= ad - b^2. \end{aligned}$$

行列式为:

$$\det(\mathbf{H}) = aC_{00} + bC_{01} + cC_{02}.$$

于是:

$$\mathbf{H}^{-1} = \frac{1}{\det(\mathbf{H})} \begin{bmatrix} C_{00} & C_{01} & C_{02} \\ C_{01} & C_{11} & C_{12} \\ C_{02} & C_{12} & C_{22} \end{bmatrix}.$$

12 步长限制与数值稳定性

代码限制单次局部牛顿更新长度:

$$\|\Delta \mathbf{x}_i\| \leq \Delta x_{\max}.$$

其中：

$$\Delta x_{\max} = \text{max_step_scale} \cdot \Delta x.$$

若牛顿步过大，则按比例缩放：

$$\Delta \mathbf{x}_i \leftarrow \Delta \mathbf{x}_i \frac{\Delta x_{\max}}{\|\Delta \mathbf{x}_i\|}.$$

这一限制可避免低迭代数、大时间步或高刚度下的局部过冲。

13 9 色并行 Gauss-Seidel

程序中颜色编号为：

$$\text{color}(i_x, i_y) = (i_x \bmod 3) + 3(i_y \bmod 3).$$

因此总共有 9 种颜色。

使用 9 色的原因是程序包含两跳弯曲弹簧。如果使用普通 2×2 四色，间隔两个网格的顶点可能颜色相同，但它们又被弯曲弹簧直接连接，GPU 同时更新会产生读写冲突。 3×3 九色能保证同色顶点之间不通过本程序定义的弹簧直接相连。

并行策略

颜色内部：同色顶点并行更新，近似 Jacobi。

颜色之间：按颜色顺序依次更新，体现 Gauss-Seidel。

整体效果：既保留块下降的收敛特性，又适合 GPU 并行。

14 速度回算与阻尼

VBD 求出新位置 $\mathbf{x}_i^{n+1}$ 后，程序用位置差回算速度：

$$\mathbf{v}_i^{n+1} = \frac{\mathbf{x}_i^{n+1} - \mathbf{x}_i^n}{h}.$$

再加入数值阻尼：

$$\mathbf{v}_i^{n+1} = \gamma \frac{\mathbf{x}_i^{n+1} - \mathbf{x}_i^n}{h}$$

其中 $\gamma = \text{damping}$ ，默认值为 0.992。该阻尼主要用于抑制长时间振荡，不等同于严格的物理粘性阻尼。

15 ParaView 输出变量说明

VTP 文件相比 OBJ 更适合 ParaView，因为它除了几何面片，还保存点数据。

推荐在 ParaView 中打开 `cloth.pvd` 而不是单独打开 OBJ。PVD 文件会索引所有 VTP 时间步，从而提供稳定的动画时间轴。

16 算法优点与局限

Table 3: VTP 点数据变量

变量	数学含义	用途
velocity	v_i	查看每个顶点速度矢量。
speed	$\ v_i\ $	直接按速度大小着色。
displacement	$x_i - X_i^0$	查看相对初始形状的位移。
fixed	0/1	显示固定点位置。
vbd_color	$(i_x \bmod 3) + 3(i_y \bmod 3)$	检查 9 色 VBD 分组。

16.1 优点

- 稳定性较好：隐式 Euler 能量形式使程序可承受较大刚度和时间步。
- GPU 友好：每个顶点只需求解一个 3×3 局部系统，不必组装全局稀疏矩阵。
- 拓扑简单：核心数据结构只有位置、速度、固定点、邻接弹簧、静止长度、刚度和颜色。
- 可视化完整：VTP/PVD 输出支持速度、位移和分组变量查看。

16.2 局限

- 当前是质量-弹簧模型，不是严格连续壳单元模型。
- 弯曲项是两跳距离弹簧，不是二面角弯曲能量。
- 未包含碰撞、自碰撞、空气阻尼和接触摩擦。
- 质量采用均匀集中质量，尚未按三角形面积精细分配。

17 最终公式总览

VBD 布料求解器的核心公式

预测点：

$$\mathbf{y}_i = \mathbf{x}_i^n + h\mathbf{v}_i^n + h^2\mathbf{g}.$$

全局隐式能量：

$$E(\mathbf{x}) = \sum_i \frac{m_i}{2h^2} \|\mathbf{x}_i - \mathbf{y}_i\|^2 + \sum_{(i,j) \in \mathcal{E}} \frac{1}{2} k_{ij} (\|\mathbf{x}_i - \mathbf{x}_j\| - L_{ij})^2.$$

单顶点更新：

$$\begin{aligned} \mathbf{x}_i^{\text{new}} = & \mathbf{x}_i - \left[\frac{m_i}{h^2} \mathbf{I} + \sum_{j \in \mathcal{N}(i)} k_{ij} \left(s_{ij}^+ \mathbf{I} + (1 - s_{ij}^+) \mathbf{n}_{ij} \mathbf{n}_{ij}^T \right) \right]^{-1} \\ & \times \left[\frac{m_i}{h^2} (\mathbf{x}_i - \mathbf{y}_i) + \sum_{j \in \mathcal{N}(i)} k_{ij} \left(1 - \frac{L_{ij}}{\|\mathbf{x}_i - \mathbf{x}_j\|} \right) (\mathbf{x}_i - \mathbf{x}_j) \right]. \end{aligned}$$

其中：

$$\mathbf{n}_{ij} = \frac{\mathbf{x}_i - \mathbf{x}_j}{\|\mathbf{x}_i - \mathbf{x}_j\|}, \quad s_{ij}^+ = \max \left(1 - \frac{L_{ij}}{\|\mathbf{x}_i - \mathbf{x}_j\|}, 0 \right).$$

结语

这份 Warp GPU 布料仿真程序的核心价值在于：它用较少的数据结构和较清晰的 GPU kernel，把隐式时间积分、非线性弹簧能量、局部牛顿法和并行着色 VBD 结合起来。对于继续开发更复杂的布料或薄壳仿真，可以在此基础上逐步加入连续膜能量、二面角弯曲、自碰撞、接触约束和更精确的质量积分。